Further simplifying Rocq and Lean proofs

Proof simplification research

22 October 2026

1 Recommendation and review scope

The next runtime simplification should reuse the existing symbolic executor, not extend it with an opaque-prefix constructor. Prove one left-frame lifting lemma, execute each small local template with the current rules, and compose its result with the existing Home contract. This reduces the new semantic machinery required by the previous proposal. Formula-local capabilities and a small shared enumeration-stack module offer separate, narrower source simplifications.

The reviewed main pin is

`6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95`

There are 26 commits after the preceding reviewed pin, `57541fe0`. The comparison API returns only 300 file entries; the review therefore also uses the complete 6,957-entry verification-area tree rather than treating that change list as exhaustive.

1.1 Accepted reductions and their evidence

Phases 13 and 14 have already adopted sharing that earlier reports proposed. Phase 13 uses one implication derivation for the unrestricted and dominated calculi: the two-file total falls from 811 to 401 lines. Phase 14 shares derived rules and transitive-container arguments, removing a further 243 lines. Across the accepted project scope, nine changed sources and one removed module account for 653 fewer lines and 36,327 fewer bytes. These are maintained-source reductions, not measured compilation speedups.

The endpoint closure decreases from 179 to 178 modules. Phase 14's selected primary ledger has 190 providers, with 66 fresh and 124 reused; compatibility has 70 providers, with 16 fresh and 54 reused. The union covers 198 distinct project modules. It is not a fresh 198-module build. The two endpoint audit logs contain ten named reports, no audit warnings and no `sorryAx`; the principal endpoint statements retain `propext`, `Classical.choice` and `Quot.sound`. One earlier failed profile API probe remains in the archive but is excluded from the accepted ledger; the corrected probe is the selected provider.

Owned replay matches all 198 current source files to the accepted source map and all 270 phase-13 and 309 phase-14 indexed archive members to their recorded hashes. It checks selected ledgers and endpoint raw logs. This establishes current-byte identity with recorded evidence, not a local kernel compilation, live-object validity check or independent replay of all inherited provenance.

The two newest K217 commits separately shorten the dominated axiom routine and generalize enumeration to start between candidates. Their commit-reported local builds are not a new full `KEndpoint217` validation. The K215 runtime, seed and dense bridge sources inspected here are unchanged. So are the inspected Rocq source, enumerator and dominated-implication files. No fresh production Rocq endpoint receipt is inferred from the Lean work.

Primary sources: phase 13 checkpoint, phase 14 checkpoint, K217 macro change and K217 enumeration change. Detailed source and receipt maps are included in the packet.

2 Reuse local symbolic runs and finish with Home

The new observation is stronger than merely grouping residual guards. Every selected local control run stops at exactly $(2, \text{true}, [], R')$: Home state 2, head on the left, and an *empty explicit left fragment*. The preserved program-counter prefix can therefore be appended after the run. Home then scans that prefix using its existing theorem, beginning at zero. This avoids the boundary-crossing argument that the previous opaque-segment design required.

2.1 The universal proof has a small explicit boundary

For a symbolic frame F, define

$$A_F(q, o, l, r) = (q, o, l +\!\!+ F, r).$$

The proposed algebraic lemma states that a successful existing `sstep` commutes with A_F, preserving its result and progress Boolean, provided the input is not head-on-left with an empty left list. This excluded case is essential: the unframed rule inserts a blank, whereas the framed configuration would read the appended data. Every other successful branch either preserves or moves only an explicit leading segment; the proposed proof splits the original cases and uses append associativity.

A fixed-length `runLocal` guards this one unsafe case before each existing step. Induction using the frame lemma and existing `sstep_sound` then gives ordinary `Reach` between the framed configurations. The final existing Home theorem supplies `Reach1`, so local positive progress is unnecessary. This is a composition of existing semantics, not a new globally trusted executor.

For one candidate template, let S be its nonempty literal input suffix, E its literal output suffix, and q its exact routed primitive state after the explicit native-to-formal label translation. Quantify over every finite prefix P satisfying

```
KHome.scanBound 0 (P.reverse.map Seg.bit) = some k
```

for some k, every natural n, and every valid register file including its nonempty spare tail. With $g = n+16$, the whole-configuration contract starts with state q, left list $\text{reverse}(P +\!\!+ \text{init}(S)) +\!\!+ [1]$, and right list beginning with the final bit of S, then 0^g, the register marker and the register file. It finishes at dispatcher state 84 with left marker [1] and right list

$$P +\!\!+ E +\!\!+ 0^{\,g+|S|-|E|} +\!\!+ [1] +\!\!+ \text{sepsR}(vs).$$

The far-left exterior remains blank. No theorem about an arbitrary nonblank exterior, only a local window, or merely equal register values is substituted for this exact tape contract.

At the empty-left endpoint, append $\text{reverse}(P)$ and the existing guard one. Existing `scanBound_sound`, the guard reset and `home_word` suffice. In particular, P need not end in one. Of the 1,422 admitted source occurrences, 721 have prefixes ending in zero. Normalize only the right endpoint with the existing normalizer: applying `normL` to an unframed left fragment could erase trailing zeros that become significant after appending the frame.

2.2 The finite evidence and the obligations it does not replace

The owned diagnostic constructs **35 candidate templates for 1,422 control guards**: 1,326 continue guards and 96 break guards. Their local bodies total 444 symbolic operations, with a maximum of 39.

The remaining three control guards are `001/CB1`, `1000111/CC3` and `11011/CB1`.

The new admitted occurrence compared with the prior family is `010111/CC3`. All 1,900 increment guards, 2,850 decrement outcomes and the halt guard stay literal. Thus the candidate expensive simulation inventory is

$$35 + 3 + 1900 + 2850 + 1 = 4789,$$

42 fewer than the prior 4,831 and 1,387 fewer than the all-literal 6,176, about 22.46%. The larger improvement is the reduced proof surface: no opaque `Seg` constructor, changed interpretation, changed normalizer, global executor extension, new arithmetic oracle or new unbounded Home invariant.

All 6,176 cheap source-coverage instances remain. Each templated occurrence must check its routed state, operation, suffix, exact prefix scan condition and gap. Continue by k requires enough ones in S and the exact stripped suffix E. Break by k requires enough zeros in S, stripping confined to that suffix, and a non-overflowing increment yielding E. These conditions prove the source action on $P +\!\!+ S$ is $P +\!\!+ E$ for every admissible P; agreement on the observed endpoints alone would not. The original gap is instantiated by $n' = n + 48 - |pc|$, using the checked bound $|pc| \le 48$, so $n' + 16 = n + 64 - |pc|$.

The finite label audit explicitly maps native states $301 \mapsto 102$, $302 \mapsto 120$ and halt $254 \mapsto 230$. It checks all 430 transition rows and 4,751 source ports, then translates and replays all 444 local steps; native 302 does occur in these bodies. Use the supplied `FORMAL_TEMPLATES.json` for Lean integration, not the raw native traces.

The replay accounts for every control occurrence, checks 2,220 finite append-frame instances and eight boundary/coverage tests, including rejection of an unsafe empty-left continuation, wrong target, wrong gap, wrong port and an eight-zero prefix. It also accepts a valid prefix ending in zero. These checks support the construction and catch implementation mistakes; they do not prove the universal frame lemma, the quantified tape contract or optimality of the 35-template partition.

The existing decoder-residual contribution already has published component evidence for all 4,751 leaves and 6,176 outcomes, not just the old two-outcome example. Its receipt reports 51 checked Lean modules and expressly excludes a full ZFC endpoint rebuild. The maintained endpoint still consumes `KSim.Covered`. Integrate through that separately checked semantic-coverage interface and an endpoint adapter. A semantic reachability contract does not establish the maintained Boolean `checkEdge = true` at its fixed fuel, so it cannot simply replace an existing literal check. No maintained family of 6,176 proofs has yet been removed. Neither the earlier semantic coverage theorem nor its component receipt is a new result of this round.

Rocq should prove the same append-frame fact and bounded-run induction in its own zipper semantics and compose its own Home theorem. Share the finite templates and exact coverage data, not a cross-kernel tape bridge. The supplied Lean theorem bodies are uncompiled sketches; the Rocq statement is an implementation plan, not a newly checked Rocq module.

3 Share named proofs through formula local capabilities

Phase 14 already shares a substantial named-calculus interface. The additional proposal finishes sharing the remaining Leibniz induction and specialization argument without giving the restricted calculus unrestricted equality vacuity. It factors the exact resource each proof consumes, rather than adding a stronger global axiom schema.

Write E_{xy} for the object-language equality formula and T for the current theorem predicate. The common Leibniz induction takes the existing propositional and row interfaces, symmetry and transitivity, the original conditions that x and y are not bound in f, and only this formula-local capability:

$$\forall i, \quad \mathrm{Bound}(i, f) \implies T(E_{xy} \Rightarrow \forall i\, E_{xy}).$$

Only the quantified case consumes that capability. In the unrestricted calculus, ordinary fresh equality vacuity supplies it. For a quantifier-free formula, there are no bound labels, so the premise is vacuous. The

restricted public wrapper still requires both `QF` and `EqFree`; the proposal deliberately does not broaden its API merely because this particular induction could handle more formulas.

Reflexivity needs even less: one explicit diagonal witness

$$T(E_{xx} \Rightarrow \forall c\, E_{xx}).$$

The unrestricted wrapper uses $c = x + 1$ with its existing freshness proof. The restricted wrapper uses its existing encoded helper label and `helper x x`. No fresh-variable axiom or general equality-vacuity field is added to the restricted interface.

The specialization bridge accepts a forward rewriting proof $T(E_{xy} \Rightarrow (f \Rightarrow g))$ and the exact elimination proof $T((\exists x\, g) \Rightarrow g)$. It derives $T((\forall x\, f) \Rightarrow g)$ from the existing rows. Callers retain responsibility for freshness and fragment conditions; the restricted caller still needs equality-freeness to construct its elimination proof. Keeping this boundary prevents the shared induction from silently strengthening the calculus.

This is a maintainability refinement, not a large size win. Counting all new helpers and compatibility wrappers, the three-file proposal changes 780 lines to 775 and 33,252 bytes to 32,595: **five lines and 657 bytes saved**. The larger per-file reduction in `KThmK` must not be reported without the adapter growth elsewhere. The short symmetry and transitivity bodies remain local because sharing them would cost more wrapper code than it removes.

Compile the three candidate files and unchanged restricted/unrestricted clients before adoption. In particular, check exact public signatures, equality-free specialization, helper-only vacuity and all existing assumption reports. The proposed generic statement is not itself a new kernel-checked theorem. Rocq can use the same resource-oriented design when a real duplicate is identified, but this packet does not claim or include a matching new Rocq calculus deletion.

4 Factor only the duplicated enumeration interfaces

4.1 Lean shares stack semantics without a generic machine framework

The dominated and command-permuted enumerators still repeat stack operations and invariant lemmas. A new `EnumStack` module extracts fifteen declarations: thirteen have byte-identical bodies; the other two reuse the shorter already-existing proofs from `EnumPerm`. Each client adds one import and transparent exported aliases, retaining the original names. The decoder, state representation, scheduler and lane-specific semantic arguments remain in their current modules.

The new shared module has 70 lines. `EnumDom` falls from 711 to 655 and `EnumPerm` from 545 to 492, so the combined total falls from 1,256 to 1,217: **39 lines and 1,361 bytes saved**, including the new module and wrapper overhead. This is separate from the already-adopted generalized K217 enumerator and from earlier scheduler proposals.

The source audit checks exact input Git blob identities and proves textually that all non-extracted client content is unchanged. That does not establish elaboration compatibility. The first acceptance tests should compile the new module, both clients and their unchanged users, especially `L2RootDom`, whose unfolding proof is a reason not to generalize the decoder in the same patch. Exported aliases are intended to preserve source-name and definitional behavior; the compiler must confirm it. This is not binary compatibility for

old object files, and metaprograms that inspect declaration identities need separate checking. The packet includes an uncompiled compatibility client.

4.2 Rocq proves each decoded step once

The maintained Rocq soundness file separately analyzes the opcode in invariant preservation and in the halting case. The proposal proves the complete one-step outcome instead:

```
Lemma step_sound s : Inv s ->
  match enum_step s with
  | Some s' => Inv s'
  | None => thm222 1
  end.
```

The existing `Inv_step` and `halt_step` statements become wrappers that rewrite the supplied step equation in this result. The existing `Inv_run` and `enum_sound` proofs stay byte-for-byte unchanged. This shares one concrete opcode analysis, not an abstract scheduler or a replacement completeness argument.

A separate five-line cleanup replaces the custom induction for `popn_S` with `Nat.iter_succ_r`, leaving the definition of `popn` unchanged. The joint-step change saves four lines; together the file decreases from 98 to 89 lines and from 4,005 to 3,648 bytes. Thus the **nine-line, 357-byte saving** is exact for the uncompiled patch, not a new Rocq receipt. The exact lemma is documented in Coq 8.18 PeanoNat; compile the source and unchanged callers under the intended production toolchain before accepting it.

The maintained Rocq file is generated from the hand-maintained `rocqperm` source. The patch updates both copies and the generated hash header. Its full source-tree change is therefore 18 lines and 714 bytes, but that is *one* nine-line proof simplification appearing twice, not two independent savings. The owned parser checks the generator's exact namespace-substitution and header relation without executing it. Production regeneration must refresh the corresponding receipt and rebuild the closure.

5 Further research questions

1. Can the frame lemma and fixed-run induction be compiled without changing any existing `sstep`, `Seg`, normalization or Home definition? This is the decisive first runtime gate, before generating all kernel certificates.

2. Does checking 35 local runs plus exact source instances reduce total kernel time and memory compared with the inherited 6,176 literal outcomes? Measure complete clean builds and object sizes, not just the number of templates. A smaller partition is not automatically faster.

3. Do the shared stack aliases preserve the definitional reductions used by unchanged lower-level clients? Keep the decoder and scheduler untouched until that narrow extraction passes.

4. Does formula-local equality capability sharing meaningfully reduce maintenance effort despite saving only five production lines? Compile the explicit audit clients, including the stronger auxiliary quantifier-free rewrite corollary, without weakening specialization's equality-free requirement.

5. Which already-proposed semantic coverage, direct-seed or quotient adapter is now cheapest to integrate into an unchanged endpoint? Those remain important integration priorities, but are not relabeled as new discoveries here.

6 Acceptance and reproduction

Prefer separate, reviewable changes. The stack and single-step patches are small source cleanups; the runtime frame lemma is the higher-value new semantic direction; the formula-local extraction is optional. Do not combine their proposed source savings with the 653 lines already removed by accepted phases 13 and 14.

No Lean or Rocq compiler is available in this environment. All new proof sources in the packet are proposals, not newly kernel-checked results. Received generators, proof modules and verification scripts were read as evidence and were not executed. The reproducible checks run only the owned offline parsers, finite diagnostics and source transformations included in the packet.

The packet contains pinned source snapshots, exact patches and proposed full source files, the uncompiled frame contract, detailed review notes, deterministic finite data and owned replay scripts. Its manifest binds every packaged file. From a fresh extraction, with Python assertions enabled and a work directory that does not yet exist, run

```
python -I -B verify.py --work /tmp/round13-check
```

The verifier checks the manifest, copies the working components outside the packet, replays current-byte and receipt checks, reconstructs the proposed patches, and reruns the finite template and boundary diagnostics. It compares deterministic results with the packaged records. The reported 62 unchanged explicit Lean theorem headers are a textual compatibility check, not proof that implicit section parameters or elaborated types match. The 32 Boolean implication cases do not verify quantified formulas.

For adoption, compile each candidate and its unchanged clients under the pinned toolchain, record exact source/import/object bindings, rebuild the affected full endpoint and compatibility scopes, and compare their named assumption reports. The runtime route additionally needs all 35 local equations, source-action framing lemmas and complete exact coverage. Recheck producer/output synchronization where files are generated. Only then measure net maintained source, build time and memory. No smaller machine, reduced endpoint axiom set, fresh cross-language endpoint receipt or measured speedup is claimed.